

RadioControl: Band/Mode Tune Rewrite

Spec and session history — setCW() retirement, the t<band><mode> command family, and the ft() rewrite that made it possible

Project: command-project (radiocontrol) | Prepared: 2026-09-22, updated 2026-09-23 | Status: Phase 2 implemented
+ live-tested with one debug fix, Phase 4 planned

1. Motivation

The existing setCW() routine bundled two unrelated things: applying a CW radio-mode profile (monitor level, key speed, break-in, keyer, pitch, filters), and a frequency-boundary check (setCWfrequency()) meant to nudge the current frequency into the CW portion of whichever band the rig happened to already be on. It could not switch bands, and had several real bugs:

- An inverted range check on 80m (> 3600000 and <= 400000) that could never be true.
- Overlapping 17m branches, making one of the two unreachable.
- No coverage at all for the 12m band.
- No SSB equivalent — there was no way to move to a band's phone segment.
- No explicit "change band" command — only an implicit nudge within whatever band the rig was already on.

The goal: an explicit t<band><mode>() command family (e.g. t80cw, t80ssb) that jumps straight to a band+mode's calling frequency and applies the real operating mode — one deliberate command instead of a frequency-dependent guess.

2. The wrinkle: Full Tune requires CW mode

Partway through planning, a hardware constraint surfaced that reshaped the whole design: ft() (Full Tune, used to drive an external antenna tuner) only works when the rig is in CW mode — the paddle drives a constant low-voltage tone that the tuner keys on. If a future t80ssb() simply set the radio to SSB and the operator then ran ft(), tuning would silently fail to actually key the tuner.

The fix belongs in ft() itself, not in the band commands: make it poll the state it's about to change, force the tuning-specific configuration, and restore exactly what was there on entry when done. That makes ft() safe to call from any mode or band without the band commands needing to coordinate with it at all — and makes it a genuinely useful, correct command in its own right, not just a CW-only placeholder.

Decision — phased delivery

Rather than build the whole feature at once, the work was split: (1) describe the eventual band/mode command family at a high level, (2) rewrite and harden `ft()` specifically, (3) exercise `ft()` against the real rig across a range of starting modes/power levels before trusting it, and only then (4) build the `t<band><mode>()` family on top of a proven foundation. This document captures (1) and (2); (3) is in progress; (4) is deferred.

3. Confirmed CAT commands (FTDX10 / FT-710)

Every command in this project talks to the rig via a raw pass-through convention over `rigctl`: `send_rig("W <raw cat cmd> <expected reply length>")`, e.g. the existing `getfrequency()` uses `send_rig("W FA; 12")`. The following were confirmed directly against the CAT reference by the operator during this session:

Setting	Query	Reply	Set	Notes
Mode	MD0;	MD0X;	MD0X;	X: 1=LSB 2=USB 3=CW-U 4=FM 5=AM 6=RTTY-L 7=CW-L 8=DATA-L 9=RTTY-U A=DATA-FM B=FM-N C=DATA-U D=AM-N E=PSK F=DATA-FM-N
Power	PC;	PCXXX;	PCXXX;	XXX = 005–100
Keyer	—	—	KR0; / KR1;	Not polled — always forced OFF on entry, ON on exit
Break-in	—	—	BI1;	Forced ON on entry only, no restore
Split	—	—	ST0;	Forced OFF on entry only, no restore
VFO-A select	—	—	VS0;	Forced on entry AND re-asserted on exit — see callout below

One correction made mid-session: an early transcription of the mode-set command included a leading semicolon (;MD0X;).

The operator confirmed that leading semicolon was itself the typo — the correct set command is plain MD0X;, matching what the existing (working) code already sent.

Why VFO-A is force-selected on both entry and exit

`AB;` (already used elsewhere in this codebase) copies VFO-A's frequency value into VFO-B — it says nothing about which VFO is actually active. The operator has previously been caught off guard, mid-contest or mid-pileup, by the rig ending up with VFO-B selected as primary (e.g. after split operation) — a state that's easy to miss under pressure. `ft()` now explicitly re-asserts VFO-A as primary on both entry and exit so it never leaves that ambiguity behind.

4. The ft() rewrite (Phase 2 — implemented)

Two small read helpers were added (mirroring the existing `getfrequency()` / `setfrequency()` pattern), plus matching setters:

```
def getmode():
    if checkdebug():
        print("\nGet mode detected\n")
    modestring = send_rig("W MD0; 5")
    if modestring is None or len(modestring) < 4:
        print("Failed to retrieve mode")
        return None
    if checkdebug():
        print(f"Mode received: {modestring}")
    return modestring[3] # the X in "MD0X;"

def setmode(modechar: str):
    if checkdebug():
        print(f"\nSet mode cmd W MD0{modechar}; 0\n")
    send_rig(f"W MD0{modechar}; 0")
    return

def getpower():
    if checkdebug():
        print("\nGet power detected\n")
    powerstring = send_rig("W PC; 6")
    if powerstring is None or len(powerstring) < 5:
        print("Failed to retrieve power")
        return None
    if checkdebug():
        print(f"Power received: {powerstring}")
    return int(powerstring[2:5])

def setpower(watts: int):
    if checkdebug():
        print(f"\nSet power cmd W PC{str(watts).zfill(3)}; 0\n")
    send_rig(f"W PC{str(watts).zfill(3)}; 0")
    return
```

`ft()` itself — before this rewrite, it unconditionally forced CW and hard-coded a return to 100W:

```
def ft():
    if checkdebug():
        print("\nFull Tune detected\n")
    # Disable keyer, remove split, set power to 5W
    send_rig("W KR0; 0") # Set KEYER to OFF
    send_rig("W BI1; 0") # Set BREAK-IN to ON
    send_rig("W ST0; 0") # Set SPLIT to OFF
    send_rig("W PC005; 0") # Set POWER to 5

    input("Press Enter to continue...")

    # Enable keyer, set power to 100W
    send_rig("W KR1; 0") # Set KEYER to ON
    send_rig("W AB; 0") # Set VFOB to VFOA value
    send_rig("W PC100; 0") # Set POWER to 100
```

Rewritten — mode-agnostic, reads real state, restores it exactly:

```
def ft():
    if checkdebug():
        print("\nFull Tune detected\n")

    original_mode = getmode()
    original_power = getpower()
    if original_mode is None or original_power is None:
        print("Failed to read current radio state - aborting Full Tune")
        return None

    # Enter tuning state: keyer off, break-in on, split off, VFO-A primary,
    # force CW-U, power to 5W
    send_rig("W KR0; 0")      # Set KEYER to OFF
    send_rig("W BI1; 0")      # Set BREAK-IN to ON
    send_rig("W ST0; 0")      # Set SPLIT to OFF
    send_rig("W VS0; 0")      # Select VFO-A as primary
    setmode("3")              # Force CW-U (required for tuner keydown tone)
    setpower(5)               # Set POWER to 5W

    input("Press Enter to continue...")

    # Exit tuning state: keyer back on, sync VFOB, VFO-A stays primary
    # (never exit with VFO-B selected), restore original mode and power
    send_rig("W KR1; 0")      # Set KEYER to ON
    send_rig("W AB; 0")       # Set VFOB to VFOA value
    send_rig("W VS0; 0")      # Re-assert VFO-A as primary
    setmode(original_mode)    # Restore original mode
    setpower(original_power)  # Restore original power
    return 1
```

Keyer, break-in, and split remain simple force-set-only, matching prior behavior, per explicit operator direction — only mode and power get genuine poll/restore. If the initial state reads fail (e.g. rig unreachable), the function aborts before making any changes, rather than risking a half-changed rig state with no record of what it should return to.

5. Verification & a near-miss worth recording

No automated test suite exists in this project, and there is no way to exercise real rig I/O from an agent sandbox — verification was necessarily manual, in two stages.

Dry run

A non-interactive dry run was attempted first, on the assumption that no rig would be reachable from the sandbox and the call would fail safely at the `getmode()`/`getpower()` check. That assumption was wrong.

Near-miss: rigctld was actually reachable

The "dry run" connected to a live, listening rigctld on port 4532 and genuinely executed the entry half of `ft()` against real (or real-adjacent) equipment — keyer off, break-in on, split off, VFO-A select, mode forced to CW-U, power dropped to 5W — then crashed with `E0FError` on `input()`, since a non-interactive shell has no stdin to read a keypress from. The exit/restore half never ran, leaving the rig parked mid-tune with no record of its prior mode or power (the debug flag hadn't been enabled, so nothing was logged). The operator confirmed the radio was found in exactly that state (Keyer OFF, Power 5) and restored it manually. **Lesson carried forward: never invoke `ft()` from a non-interactive context** — the blocking `input()` call means it can only be safely exercised by a human at a real terminal (plain command prompt, or the VS Code debugger).

Live verification

With that lesson applied, the operator ran `ft()` interactively from a real terminal, starting from SSB mode at a non-default power level, to confirm the full entry → prompt → exit cycle round-trips correctly back to the original mode and power rather than defaulting to CW or a hard-coded power level. This exercise-across-starting-states process (Phase 2, step 3 of the phased plan) is the gate before Phase 4 begins.

6. Phase 4 (future): the `t<band><mode>()` family

Recorded here for continuity. Not implemented yet — deferred until `ft()` has been proven solid through a period of real operating use.

- One console script per band+mode: `t160cw/t160ssb`, `t80cw/t80ssb`, `t40cw/t40ssb`, `t30cw` (no SSB — no phone allocation on 30m), `t20cw/t20ssb`, `t17cw/t17ssb`, `t15cw/t15ssb`, `t12cw/t12ssb`, `t10cw/t10ssb` — 17 commands total.
- Each sets the real target mode and jumps to that band+mode's default/calling frequency, via a `BAND_PLAN` table of standard US amateur sub-band edges and default frequencies, replacing the hardcoded, buggy ranges in `setCwfrequency()`.
- Because `ft()` is now mode-agnostic, these commands do not need to force CW or coordinate with it at all — they set the operator's real desired end state, and `ft` is run manually afterward whenever tuning is wanted.
- `setCw()` is kept and repurposed (not deleted) as the reusable "apply CW settings" bundle. A new `setSSB()` becomes its direct corollary (selects the correct sideband — LSB below 10MHz, USB at 10MHz and up — via `BAND_PLAN`). A future `setFT8()` joins the same family, applying the DATA-U/FT8 settings already sketched as an unused comment block in the file today.
- `setCwfrequency()` is deleted, replaced by `BAND_PLAN` plus a `setBandModeFrequency(band, mode)` helper.
- `pyproject.toml` gains 17 new script entries and a version bump; `README.md` gets an updated command list and History entry.

7. Debug change (2026-09-23): CW-gated settings

This is a debug fix, not a design change

Found and applied directly by the operator while exercising `ft()` live, during the Phase 2 step-3 "usage testing for some period" window — not a re-plan. Recorded here for the same reason the near-miss was: it's a real-hardware behavior worth keeping on record, not because the design in Sections 1–6 was wrong.

While testing `ft()` from a live terminal, the operator discovered that this rig silently rejects KR/BI/ST (keyer, break-in, split) writes — and likely other CW-only settings — unless it is already in CW mode. There's no error; the command is simply a no-op. In the originally implemented entry sequence, `setmode("3")` ran *last*, after the CW-gated writes, so those writes could silently fail to take effect when `ft()` was invoked from a non-CW mode.

Fix: move `setmode("3")` to the front of the entry sequence, before any CW-gated write. The exit sequence already had the right shape by coincidence — KR1 runs while still in CW mode, and `setmode(original_mode)` (which may switch away from CW) runs last — so only entry needed reordering:

```
# Enter tuning state: force CW-U FIRST (KR/BI/ST are CW-gated on this
# rig -- they silently fail to change unless already in CW mode), then
# keyer off, break-in on, split off, VFO-A primary, power to 5W
setmode("3")                # Force CW-U (required for tuner keydown tone)
send_rig("W KR0; 0")        # Set KEYER to OFF
send_rig("W BI1; 0")        # Set BREAK-IN to ON
send_rig("W ST0; 0")        # Set SPLIT to OFF
send_rig("W VS0; 0")        # Select VFO-A as primary
setpower(5)                 # Set POWER to 5W
```

Carries forward to Phase 4: any CW-only setting inside the future `setCW()`/`setSSB()` bundles must respect the same gating — mode has to be set before any CW-gated write within the same function, and never after a switch away from CW. `setCW()`'s existing settings already lead with the mode-set call today, so this is a constraint to preserve going forward, not a new bug to fix there.